

Operadores, Tipos de Datos y Técnicas Avanzadas de Manipulación en C

Pedro Nicolás Petrashin
Instituto Universitario Aeronáutico
npetrashin@iua.edu.ar

Resumen

Este trabajo presenta una revisión de los principales operadores y estructuras del lenguaje C relacionados con la manipulación de bits y tipos de datos. Se abordan operadores lógicos y de desplazamiento, el manejo de tipos signados y no signados, la librería <stdint.h>, así como campos de bits, uniones y enumeraciones. Se incluyen ejemplos de aplicación en la decodificación de tramas de comunicación como RS232 y GPS, y en el cálculo de códigos de verificación (CRC).

1. Introducción

El lenguaje de programación C se caracteriza por ofrecer un acceso directo y eficiente a los recursos del hardware, lo que lo convierte en una de las herramientas más utilizadas en el desarrollo de sistemas embebidos, controladores de dispositivos, protocolos de comunicación y aplicaciones que requieren un manejo preciso de datos a bajo nivel. A diferencia de lenguajes de más alto nivel, C permite manipular directamente la representación binaria de los datos mediante operadores lógicos y de desplazamiento, así como estructuras especializadas para optimizar el uso de memoria y mejorar el rendimiento.

El conocimiento y dominio de conceptos como los operadores de bits, los tipos de datos signados y no signados, y el uso de librerías estándar como <stdint.h> resulta esencial para diseñar aplicaciones portables y seguras. Asimismo, las herramientas provistas por el lenguaje, tales como campos de bits, uniones y enumeraciones, permiten estructurar la información de forma clara y eficiente, adaptándose a las necesidades específicas de cada aplicación.

Estas técnicas son ampliamente utilizadas en la decodificación de tramas de comunicación (como RS232, GPS o enlaces satelitales), en la configuración de registros de hardware y en la implementación de algoritmos de

verificación como los códigos de redundancia cíclica (CRC). En todos estos casos, la capacidad de manipular datos a nivel de bit representa una ventaja significativa en términos de rendimiento, portabilidad y control del sistema.

En el contexto de la programación de bajo nivel, uno de los desafíos más frecuentes consiste en representar y manipular datos de manera eficiente, maximizando la precisión y minimizando el uso de memoria. La necesidad de acceder directamente a bits individuales surge en aplicaciones como la transmisión serial, la lectura de sensores y la codificación de mensajes compactos.

Por tanto, el objetivo de este trabajo es presentar y ejemplificar los mecanismos que ofrece el lenguaje C para operar a nivel de bits, y demostrar cómo estos recursos permiten optimizar la comunicación de datos y mejorar la comprensión de la arquitectura de los sistemas embebidos.

2. Marco Teórico

El lenguaje C es una de las herramientas más utilizadas en el desarrollo de sistemas embebidos, firmware y software de bajo nivel, debido a su capacidad de manipular directamente la memoria y representar datos a nivel de bits. En este contexto, el conocimiento de los operadores de bits, los tipos de datos y las estructuras especiales del lenguaje resulta esencial para un control eficiente del hardware y para la comunicación entre dispositivos [1].

3. Operadores de Bits

Los operadores de bits permiten manipular de manera directa la representación binaria de los datos [2]. Su uso es fundamental en aplicaciones donde cada bit tiene un significado particular, como registros de hardware, banderas de estado o protocolos de comunicación. A continuación, presentamos los diferentes operadores:

AND (&):

Compara cada bit de dos operandos, devolviendo 1 solo si ambos son 1. Se utiliza comúnmente con **máscaras** para aislar ciertos bits:

Ejemplo:

```
uint8_t valor = 0b10101100;    // 172
en decimal
uint8_t mascara = 0b00001111;  //
aísla los 4 bits menos significativos
uint8_t resultado = valor & mascara;
// resultado = 0b00001100 (12)
```

Fig.1: representación en C de AND

OR (|):

Devuelve 1 si al menos uno de los bits es 1. Sirve para **activar** ciertos bits sin modificar los demás:

Ejemplo:

```
uint8_t config = 0b00000001;  // bit
0 activo

config = config | 0b00000100;  //
activa también el bit 2

// config = 0b00000101
```

Fig.2: representación en C de OR

XOR (^):

Devuelve 1 solo si los bits son diferentes.

Se emplea para invertir bits específicos o en algoritmos de verificación como CRC:

Ejemplo:

```
uint8_t estado = 0b10101010;
uint8_t mascara = 0b11110000;
uint8_t resultado = estado ^ mascara;
// invierte los 4 bits más altos
// resultado = 0b01011010
```

Fig.3: representación en C de XOR

Complemento (~):

Invierte todos los bits del operando:

```
uint8_t valor = 0b00001111;
uint8_t resultado = ~valor;    //
resultado = 0b11110000
```

Fig.4: representación en C del complemento

Por ejemplo, supongamos que recibimos un byte de estado en un sistema embebido con la siguiente codificación:

- Bit 0: error de paridad
- Bit 1: overflow
- Bit 2: transmisión completa
- Bits 3-7: reservados

uint8_t estado = 0b00000101; // transmisión completa + error de paridad

Ejemplo:

```
if (estado & 0b00000001) {
    // Error de paridad detectado
}
if (estado & (1 << 2)) {
    // Transmisión completa
}
```

Fig.5: ejemplo de uso del AND y Corrimiento

Aquí se combinan **AND** y **corrimientos** para verificar banderas individuales sin afectar el resto del byte.

4. Operadores de Desplazamiento

En C existen dos operadores de desplazamiento de bits:

- Desplazamiento a la izquierda (<<)
- Desplazamiento a la derecha (>>)

Estos operadores permiten mover bits dentro de una variable, lo que es especialmente útil en manipulación de registros, optimización de operaciones aritméticas y codificación/decodificación de tramas [3].

Desplazamiento a la izquierda:

Desplaza todos los bits hacia la izquierda y rellena con ceros por la derecha. Es equivalente a multiplicar por potencias de 2, siempre que no haya overflow.

Ejemplo:

```
#include <stdio.h>
#include <stdint.h>
int main() {
    uint8_t valor = 5;           //
    00000101 (5 en binario)
    uint8_t resultado = valor << 2;
    // Desplaza dos posiciones
    printf("Resultado:          %d\n",
resultado); // 20
    return 0;
}
```

Fig.7: ejemplo de desplazamiento a la izquierda en C

Uso típico:

- Construcción de máscaras de bits.
- Desplazamiento de datos para empaquetarlos en tramas.
- Multiplicaciones rápidas en microcontroladores sin FPU.

```
uint8_t mask = 1 << 5; // 0b00100000
```

Fig.7: Ejemplo de creación de una máscara para el bit 5

Desplazamiento a la derecha:

Desplaza todos los bits hacia la derecha. En tipos sin signo introduce ceros; en tipos con signo puede replicar el bit de signo (extensión aritmética).

Ejemplo:

```
uint8_t valor = 0b10101000;
uint8_t resultado = valor >> 3; //
0b00010101 (21 en decimal)
```

Fig.8: ejemplo de desplazamiento a la derecha en C

Uso típico:

- Aislar partes de un valor (por ejemplo, el byte más alto de un entero de 16 bits).
- Dividir por potencias de 2 sin usar operaciones costosas.

Combinación con Operadores Lógicos:

Los desplazamientos suelen combinarse con AND, OR y XOR para leer o modificar bits en posiciones específicas.

Ejemplo: configuración de un registro activando el bit 3 y desactivando el bit 1:

```
uint8_t registro = 0b00001110;
// Activar bit 3:
registro |= (1 << 3); // registro =
0b00001110 (ya estaba activado)
// Desactivar bit 1:
registro &= ~(1 << 1); // registro =
0b00001100
```

Fig.9: ejemplo en C de la combinación de varios operadores lógicos

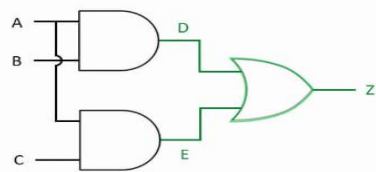


fig.10:Ejemplo del circuito lógico.

5. Tipos de Datos Signados y No Signados

En C, un entero puede ser signado (con signo) o no signado (sin signo). La elección afecta el rango de valores representables. Los no signados son útiles en operaciones con datos binarios donde no se requieren negativos.

En C, los tipos enteros pueden declararse como signados (con signo) o no signados (unsigned). Esta elección es crucial cuando trabajamos con datos binarios o cuando necesitamos rangos de valores específicos.

Tipos signados:

Los enteros signados pueden representar números positivos y negativos, usando el bit más significativo (MSB) como bit de signo:

- 0 → número positivo
- 1 → número negativo (representación en complemento a dos)

Ejemplo:

```
#include <stdio.h>
#include <stdint.h>
int main() {
    int8_t a = -5; // rango: -128 a
127
    int8_t b = 10;
    printf("Suma: %d\n", a + b); // 5
    return 0;
}
```

Fig.11: ejemplo de datos signados en C

si ocurre overflow (ej. sumar 127 + 1 en int8_t), el comportamiento es indefinido.

Tipos no signados:

Los no signados representan únicamente valores positivos, por lo que duplican el rango positivo de su equivalente signado.

Por ejemplo:

- int8_t → rango -128 a 127 (valores *signados*)
- uint8_t → rango 0 a 255 (valores *no signados*)

```
uint8_t a = 250;
uint8_t b = 10;
uint8_t suma = a + b; // Overflow →
resultado = 4 (se pierde la parte
alta)
```

Fig.12: ejemplo de datos no signados en C

Esto se conoce como overflow modular, pero en el caso de tipos unsigned el comportamiento está definido en C: el resultado se calcula módulo 2^n .

Comparaciones y Mezcla de Tipos:

Si se comparan un valor signado con uno no signado, el valor signado se convierte implícitamente a no signado. Esto puede llevar a resultados inesperados:

```
int8_t x = -1;
uint8_t y = 1;
```

```
if (x < y) {
    printf("x es menor que y\n");
}
```

Fig. 13: ejemplo en C de la mezcla de tipos

Aquí x se convierte a uint8_t, dando 255, por lo que la comparación es falsa.

Aplicaciones:

- Tipos signados: útiles en operaciones aritméticas, índices que pueden ser negativos (desplazamientos), cálculos matemáticos generales.
- Tipos no signados: recomendados para manipulación de datos binarios (máscaras, registros de hardware, buffers de comunicación), ya que evitan problemas de extensión de signo.

6. Librería stdint.h

La librería <stdint.h> forma parte del estándar C99 y proporciona tipos enteros de tamaño fijo, asegurando que el ancho (cantidad de bits) sea el mismo en todas las plataformas. Esto resuelve el problema de que tipos como int o long pueden variar en tamaño según el compilador o la arquitectura (16, 32 o 64 bits).

Tipos Enteros de Ancho Fijo:

int8_t: Entero con signo de 8 bits, de -128 a 127

uint8_t: Entero sin signo de 8 bits, de 0 a 255

int16_t: Entero con signo de 16 bits, de -32768 a 32767

uint16_t: Entero sin signo de 16 bits, de 0 a 65535

int32_t: Entero con signo de 32 bits, de -2,147,483,648 a 2,147,483,647

uint32_t: Entero sin signo de 32 bits, de 0 a 4,294,967,295

Estos tipos garantizan que un uint16_t siempre tenga exactamente 16 bits, sin importar si el código se compila en una PC de 64 bits o en un microcontrolador de 8 bits.

Constantes y límites:

La librería define constantes para los límites de cada tipo, por ejemplo:

```
#include <stdint.h>
#include <stdio.h>
int main() {
    printf("Max      uint8_t:    %u\n",
UINT8_MAX); // 255
    printf("Min      int16_t:    %d\n",
INT16_MIN); // -32768
    return 0;
}
```

Fig. 14: representación en C de los límites de cada tipo de variable

Esto es preferible a escribir valores “mágicos” (como 255) directamente en el código, mejorando su legibilidad y mantenimiento.

Metodología

Ejemplo de Uso Práctico:

Supongamos que queremos representar un mensaje de sensor que puede variar de 0 a 1023 (10 bits). Para garantizar portabilidad:

```
#include <stdint.h>
typedef struct {
    uint16_t valor; // garantiza 16
bits, suficiente para 10 bits de datos
    uint8_t id; } MensajeSensor;
```

Fig. 15: ejemplo de uso práctico

Esto asegura que valor y id tengan el mismo tamaño en cualquier plataforma donde se compile.

Ventajas de Usar <stdint.h>:

- Portabilidad: mismo tamaño en cualquier plataforma.
- Claridad: el tipo indica el rango esperado.
- Seguridad: evita errores por overflow involuntario debido a cambios de tamaño.
- Compatibilidad: especialmente útil en protocolos y estructuras de comunicación.

7. Campos de Bits

Los campos de bits permiten definir estructuras en las que cada miembro ocupa un número específico de bits, en lugar de los 8, 16 o 32 que usaría normalmente un tipo entero. Esto es muy útil para representar registros de hardware o tramas de comunicación donde cada bit tiene un significado particular.

Definición de Campos de Bits

Un campo de bits se declara dentro de una struct, indicando cuántos bits ocupa cada miembro:

```
struct Registro {
    unsigned int banderal : 1; // 1
bit
    unsigned int bandera2 : 1; // 1
bit
    unsigned int codigo    : 6; // 6
bits
};
```

Fig. 16: ejemplo en C de campos de bits

En este ejemplo, la estructura ocupa un total de 8 bits (1 byte). El compilador agrupa los campos hasta llenar el tamaño del tipo base (unsigned int por defecto), aunque la alineación exacta depende de la arquitectura.

Acceso a los Miembros:

El acceso a los campos se realiza como si fueran variables normales:

```
struct Registro reg;
reg.banderal = 1;
reg.codigo = 0x2A; // 42 en decimal
```

Fig. 17: representación en C de acceso a miembros

Ejemplo de Uso Práctico

Supongamos un registro de control de un periférico donde cada bit representa una función:

- bit 0: habilita el periférico
- bit 1: habilita interrupciones
- bits 2-3: modo de operación (00 = reposo, 01 = normal, 10 = rápido, 11 = diagnóstico)
- bits 4-7: reservados
- Podemos definirlo así:

```

struct Control {
    unsigned int enable      : 1;
    unsigned int irq_enable  : 1;
    unsigned int mode        : 2;
    unsigned int reservado   : 4;
};
struct Control ctrl;
ctrl.enable = 1;
ctrl.mode = 2; // modo rápido

```

Fig. 18: representación en C del ejemplo

Esto facilita la escritura de código legible, en lugar de tener que usar máscaras manualmente.

8. Uso de sizeof, Máscaras y Corrimientos

El operador sizeof, junto con el uso de máscaras y corrimientos de bits, es fundamental para escribir código portable y eficiente en C, especialmente cuando se manipulan datos a nivel bajo. sizeof devuelve el tamaño en bytes de un tipo o variable en tiempo de compilación.

```

#include <stdio.h>
int main() {
    int x;
    printf("Tamaño de int: %zu\n",
sizeof(x));
    printf("Tamaño de long: %zu\n",
sizeof(long));
    return 0;
}

```

Fig.19: ejemplo en C de Sizeof

Esto es útil para:

- Determinar dinámicamente el tamaño de un arreglo.
- Crear estructuras de datos portables.
- Evitar errores al usar funciones como memcpy o fread.
- Ejemplo para recorrer un arreglo completo:

```

int arr[] = {1, 2, 3, 4};
size_t elementos = sizeof(arr) /
sizeof(arr[0]);

```

Fig.20: ejemplo en C de array

Máscaras de Bits

Una máscara es un valor binario usado para seleccionar o modificar bits específicos de otro valor. Se usan junto con operadores &, |, ^ y ~.

Aislar un bit específico:

```

uint8_t registro = 0b10101100;
if (registro & (1 << 3)) {
    // El bit 3 está en 1
}

```

Fig. 21: ejemplo de aislamiento de bits

Activar un bit (sin modificar el resto):

```

registro |= (1 << 2); // Activa el
bit 2

```

Fig. 22: ejemplo de activar bits

Desactivar un bit:

```

registro &= ~(1 << 2); // Desactiva
el bit 2

```

Fig. 23: ejemplo de desactivar bits

Corrimientos de Bits

Los corrimientos se usan para alinear valores o para extraer campos de un registro.

```

uint16_t dato = 0xABCD;
uint8_t msb = (dato >> 8) & 0xFF; //
0xAB
uint8_t lsb = dato & 0xFF;          //
0xCD
Ejemplo: empaquetar dos valores de 4
bits en un byte
uint8_t a = 0x0A; // 00001010
uint8_t b = 0x05; // 00000101
uint8_t empaquetado = (a << 4) | b;
// 10100101

```

Fig. 23: Ejemplo de extracción de byte

Decodificación de Mensaje

Supongamos que recibimos un byte de estado de un sensor:

- Bit 0 → error
- Bit 1 → advertencia
- Bits 2-3 → modo (0 reposo, 1 normal, 2 rápido, 3 diagnóstico)
- Bits 4-7 → código de error

Podemos decodificarlo así:

```
uint8_t estado = 0b10110101;
uint8_t error = estado & 0x01;
uint8_t advertencia = (estado >> 1) & 0x01;
uint8_t modo = (estado >> 2) & 0x03;
// aislar bits 2 y 3
uint8_t codigo_error = (estado >> 4) & 0x0F; // aislar bits 4-7
```

Fig. 24: ejemplo de decodificación

9. Uniones

Una unión en C es una estructura especial que permite almacenar diferentes tipos de datos en la misma región de memoria. A diferencia de una struct, donde cada miembro ocupa un espacio adicional, en una unión todos los miembros comparten el mismo espacio.

La sintaxis es similar a la de una estructura:

```
union Conversor {
    uint32_t entero;
    uint8_t bytes[4];
};
```

Fig. 25: ejemplo en C de unión

En este ejemplo, entero y bytes comparten los mismos 4 bytes de memoria. El tamaño de la unión será el del miembro más grande (en este caso, uint32_t).

Inicialización y Uso

Podemos asignar un valor a un miembro y leerlo desde otro:

```
#include <stdio.h>
#include <stdint.h>
int main() {
    union Conversor dato;
```

```
    dato.entero = 0x12345678;

    printf("Byte      0:      0x%X\n",
    dato.bytes[0]);
    printf("Byte      1:      0x%X\n",
    dato.bytes[1]);
    printf("Byte      2:      0x%X\n",
    dato.bytes[2]);
    printf("Byte      3:      0x%X\n",
    dato.bytes[3]);
    return 0;
}
```

Fig.26: ejemplo de uso de uniones en C

Esto es muy útil para dividir un entero en bytes individuales, por ejemplo, para enviar por UART o empaquetar en un protocolo.

Combinación con Estructuras y Campos de Bits

Podemos usar una unión junto con una estructura de campos de bits para interpretar un mismo valor de distintas maneras:

```
union Estado {
    uint8_t byte;
    struct {
        unsigned int error      : 1;
        unsigned int advertencia: 1;
        unsigned int modo       : 2;
        unsigned int codigo     : 4;
    } bits;
};
union Estado estado;
estado.byte = 0b10110101;
if (estado.bits.error) {
    printf("Error detectado\n");
}
printf("Código de error:  %d\n",
    estado.bits.codigo);
```

Fig. 27: ejemplo en C de combinación

Aquí se puede leer el valor completo (estado.byte) o acceder a cada bit individual (estado.bits.error).

Ventajas

- Permiten ahorrar memoria, ya que los miembros no se almacenan por separado.

- Facilitan la reinterpretación de datos, muy útil en protocolos y drivers.
- Combinadas con estructuras, permiten crear representaciones muy expresivas de datos complejos.

10. Aplicaciones en Comunicaciones

En sistemas embebidos y comunicaciones es muy común trabajar con tramas de datos donde cada bit tiene un significado específico. Los operadores de bits, máscaras, corrimientos, estructuras y uniones son herramientas ideales para empaquetar y desempaquetar esa información.

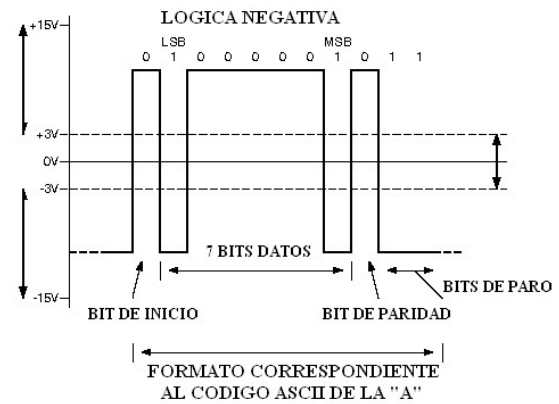


Fig.28: representación gráfica de la forma de onda

Ejemplo en C

Supongamos que recibimos un **entero de 11 bits** (por simplicidad lo guardamos en uint16_t):

Trama RS232

En una comunicación RS232, los datos se envían **bit a bit** en un orden específico. Un formato común es:

| Start | D0 | D1 | D2 | D3 | D4 | D5 | D6 | Paridad | Stop | Stop |

- **Start:** siempre 0 (indica comienzo de trama).
- **D0-D6:** 7 bits de datos (carácter ASCII, por ejemplo).
- **Paridad:** bit calculado para verificar errores.
- **Stop:** bits en 1, indican el final del carácter.

En C, podemos simular la recepción de este byte extendido y decodificarlo usando máscaras y corrimientos. (Ver Fig. 1)

En la figura 1 se observa como sería la forma de onda:

```
#include <stdio.h>
#include <stdint.h>

int main() {
    // Ejemplo de trama: Start=0,
    Datos=0b1001101 ('M'), Paridad=1,
    Stop=11
    uint16_t trama = 0b11011001101;

    // Extraer cada parte usando
    máscaras y corrimientos
    uint8_t start = trama & 0x01;
    // bit 0
    uint8_t datos = (trama >> 1) &
    0x7F; // bits 1-7
    uint8_t paridad = (trama >> 8) &
    0x01; // bit 8
    uint8_t stop = (trama >> 9) &
    0x03; // bits 9-10
    printf("Start: %u\n", start);
    printf("Datos: 0x%X (%c)\n",
    datos, datos);
    printf("Paridad: %u\n", paridad);
    printf("Stop bits: %u\n", stop);
    return 0;
}
```

Fig. 29: ejemplo en C de la aplicación

Explicación

- Usamos **máscaras** (0x7F = 01111111b) para quedarnos solo con los bits de datos.
- Desplazamos (>>) antes de enmascarar para alinear los bits al inicio.
- El bit de paridad se usa para verificar que el número de bits en 1 en toda la trama es par (paridad par) o impar (paridad impar).
- Los bits de stop se revisan para confirmar que la línea está en reposo.

Este ejemplo muestra cómo, con operaciones de **corrimiento y AND**, se pueden extraer partes de una trama y validar la integridad de la comunicación.

Cálculo de CRC (Cyclic Redundancy Check)

Es un **algoritmo de verificación de errores** que genera un pequeño valor (checksum) a partir de un mensaje, de forma que si el mensaje cambia (por errores de transmisión), el CRC también cambia.

Se usa ampliamente en:

- Comunicaciones seriales (RS232, CAN, Modbus).
- Protocolos de red (Ethernet, USB).
- Almacenamiento de datos (archivos, discos duros).

El receptor **recalcula el CRC** y lo compara con el CRC recibido:

- Si coinciden → mensaje válido.
- Si difieren → error de transmisión.

Cómo se Calcula

El cálculo de un CRC es similar a una **división polinómica binaria**:

1. Se interpreta el mensaje como un polinomio de bits.
2. Se divide por un polinomio generador predefinido.
3. El **resto** de la división es el CRC.

Los CRC se calculan usando XOR y corrimientos. Un ejemplo simple de CRC-8:

```
#include <stdint.h>
#include <stddef.h>

uint8_t calcularCRC8(uint8_t *datos,
size_t longitud) {
    uint8_t crc = 0x00;
    for (size_t i = 0; i < longitud;
i++) {
        crc ^= datos[i]; // XOR con
el dato
        for (int j = 0; j < 8; j++) {
            if (crc & 0x80) // si el
bit más significativo es 1
                crc = (crc << 1) ^
0x07; // XOR con el polinomio
            else
                crc <<= 1;
        }
    }
    return crc;
}
```

Fig. 30: ejemplo en C de CRC

Este tipo de algoritmos es fundamental para verificar la integridad de datos en transmisiones.

Ventajas de Estas Técnicas

- Permiten ahorrar memoria y ancho de banda al compactar datos.
- Facilitan la interpretación de protocolos complejos.

- Mejoran la legibilidad del código al usar estructuras y uniones.
- Garantizan exactitud en el control de hardware.

11. Enumeraciones

Las enumeraciones (enum) en C son una forma de definir conjuntos de constantes simbólicas asociadas a valores enteros. Sirven para mejorar la legibilidad y mantenibilidad del código, reemplazando números “mágicos” por nombres significativos.

Una enumeración se define con la palabra clave enum:

```
enum EstadoConexion {
    DESCONECTADO, // 0
    CONECTANDO,   // 1
    CONECTADO,     // 2
    ERROR         // 3
};
```

Fig. 31: ejemplo en C de Enum

Por defecto, el primer valor empieza en 0 y los siguientes se incrementan de uno en uno, aunque se pueden asignar valores manualmente:

```
enum EstadoConexion {
    DESCONECTADO = 0,
    CONECTANDO   = 10,
    CONECTADO    = 20,
    ERROR        = 99
};
```

Fig.32: Ejemplo 2 de Enum

Una vez definida, se puede declarar una variable de ese tipo:

```
enum EstadoConexion estado;
estado = CONECTANDO;
if (estado == CONECTADO) {
    printf("El sistema está
conectado\n");
}
```

Fig. 33: aplicacion en C de Enum

Esto es mucho más claro que usar if (estado == 2), que no da contexto al lector.

Una de las formas más útiles de emplear enumeraciones es en sentencias switch, ya que permiten cubrir todos los casos posibles de forma ordenada:

```
switch (estado) {
    case DESCONECTADO:
        printf("Sin conexión.\n");
        break;
    case CONECTANDO:
        printf("Estableciendo
conexión...\n");
        break;
    case CONECTADO:
        printf("Conexión
establecida.\n");
        break;
    case ERROR:
        printf("Error de
conexión.\n");
        break;
}
```

Fig.34: aplicacion de Enum en Switch

Enumeraciones con Máscaras de Bits

También es posible usar enumeraciones para definir máscaras de bits, lo que hace el código más descriptivo:

```
enum BanderasEstado {
    FLAG_ERROR      = (1 << 0),
    FLAG_ADVERTENCIA= (1 << 1),
    FLAG_TX_READY   = (1 << 2)
};
uint8_t estado = FLAG_ERROR |
FLAG_TX_READY;
if (estado & FLAG_TX_READY) {
    printf("Listo para
transmitir\n");
}
```

Fig.35: Ej en C de enum con mascaras de bits

De esta manera, se evita escribir estado & (1 << 2) y el código queda autoexplicativo.

Combinación con Uniones y Campos de Bits

Podemos usar enumeraciones para representar valores simbólicos dentro de un campo de bits:

```
enum ModoOperacion {
    REPOSO = 0,
    NORMAL = 1,
    RAPIDO = 2,
    DIAGNOSTICO = 3
};
union Control {
    uint8_t byte;
    struct {
        unsigned int enable : 1;
        unsigned int irq : 1;
        enum ModoOperacion modo : 2;
        unsigned int reservado : 4;
    } bits;
};
union Control ctrl;
ctrl.bits.modo = RAPIDO;
```

Fig. 36: ejemplo en C de combinacion de enum, uniones, y campos de bits

Así el valor 2 no es un número arbitrario, sino rápido, lo que facilita su comprensión y evita errores.

12. Metodología

Para demostrar la aplicación práctica de los conceptos desarrollados, se implementó un ejemplo basado en la decodificación de una trama de comunicación RS232.

La trama seleccionada tiene el siguiente formato: 1 bit de inicio, 7 bits de datos, 1 bit de paridad y 2 bits de parada, que en total conforman 11 bits.

El programa en C realiza las siguientes acciones:

- Recibe una trama de 11 bits representada en un entero sin signo.
- Aplica operaciones de máscaras y corrimientos para aislar las diferentes secciones.
- Calcula el bit de paridad para validar la integridad de los datos.
- Muestra por pantalla los valores obtenidos de cada campo.

Este procedimiento permite observar cómo el uso de operadores bit a bit, uniones y desplazamientos facilita la manipulación de estructuras de comunicación compactas sin recurrir a tipos de datos complejos.

```
#include <stdio.h>
#include <stdint.h>
// Función para calcular la paridad
(1 si número de bits en 1 es impar)
uint8_t calcular_paridad(uint8_t
dato) {
    uint8_t contador = 0;
    for (int i = 0; i < 7; i++) {
        if (dato & (1 << i))
            contador++;
    }
    return contador % 2; // Paridad
impar
}

int main() {
    // Trama ejemplo: Start=0,
    Datos=0b1001101 ('M'), Paridad=1,
    Stop=11
    // Se representa como: 11 1
    1001101 0 (de derecha a izquierda
    en transmisión)
    uint16_t trama = 0b11011001101;
    // 11 bits en total

    // Extraer los campos mediante
    máscaras y corrimientos
    uint8_t start = trama & 0x01;
    // bit 0
    uint8_t datos = (trama >> 1) &
    0x7F; // bits 1 a 7 (7 bits de
    datos)
    uint8_t paridad = (trama >> 8) &
    0x01; // bit 8
    uint8_t stop = (trama >> 9) &
    0x03; // bits 9 y 10

    // Calcular la paridad del dato
    recibido
    uint8_t paridad_calculada =
    calcular_paridad(datos);

    // Mostrar los resultados
    printf("=== Decodificación de
    trama RS232 ===\n");
    printf("Trama binaria:
    0b11011001101\n\n");
    printf("Bit de inicio (start):
    %u\n", start);
    printf("Datos (7 bits): 0x%02X
    ('%c')\n", datos, datos);
    printf("Bit de paridad: %u\n",
    paridad);
```

```

    printf("Bits de parada (stop):
    %u\n\n", stop);

    // Verificar paridad
    if (paridad ==
    paridad_calculada)
        printf("Paridad
    correcta\n");
    else
        printf("Error de paridad
    (esperado: %u)\n",
    paridad_calculada);

    return 0;
}

```

Fig.36: Código en C de la Metodología sugerida

Resultados

La implementación permitió decodificar correctamente los valores de cada campo dentro de la trama. Por ejemplo, para una trama de entrada 0b11011001101, se obtuvieron los siguientes resultados:

Campo	Bits	Valor	Descripción
Start	1	0	Inicio de transmisión
Datos	7	0x4D	'M' (carácter ASCII)
Paridad	1	1	Paridad correcta
Stop	2	3	Fin de transmisión

Fig.37: representación gráfica del output

Los resultados obtenidos evidencian que la manipulación directa de bits permite ahorrar memoria y ciclos de procesamiento, comparado con la manipulación a nivel de bytes o estructuras más complejas.

Además, el uso de uniones y campos de bits mejora la claridad y mantenimiento del código, ya que permite acceder a partes específicas de un dato sin recurrir a conversiones o punteros. Este enfoque es especialmente útil en protocolos seriales, drivers de hardware y sistemas embebidos, donde el control preciso de cada bit es esencial.

Finalmente, se observa que los conceptos abordados en el marco teórico tienen una aplicación directa y tangible en el diseño de software eficiente para comunicación y control.

13. Conclusiones

A partir de los resultados y la discusión previa, se puede concluir que el lenguaje C ofrece un conjunto de herramientas poderosas para el control de bajo nivel, que permiten interactuar directamente con el hardware y optimizar el uso de memoria y tiempo de ejecución.

A lo largo de este trabajo se revisaron conceptos fundamentales como operadores de bits, desplazamientos, tipos signados y no signados, el uso de <stdint.h>, campos de bits, máscaras, uniones y enumeraciones.

Estos recursos son la base para:

- Desarrollar drivers de hardware, ya que permiten mapear registros de control y estado.
- Decodificar y construir tramas de comunicación, extrayendo e insertando bits y bytes en posiciones exactas.
- Optimizar aplicaciones en sistemas embebidos, reduciendo consumo de memoria y ciclos de CPU.
- Implementar algoritmos de verificación, como CRC, que son esenciales para la integridad de datos.
- Escribir código portable y mantenible, gracias al uso de tipos de tamaño fijo (stdint.h) y enumeraciones descriptivas.

Más allá de su uso técnico, estos temas ayudan a desarrollar un pensamiento lógico y preciso en los estudiantes, mostrándoles cómo funciona la representación de datos a nivel binario y fomentando el razonamiento paso a paso, habilidades cruciales para un programador.

En definitiva, el dominio de estas herramientas permite escribir software más robusto, eficiente y comprensible, acercando a los estudiantes al mundo del desarrollo de sistemas embebidos, comunicación de datos y programación de bajo nivel, que son pilares de la ingeniería en computación.

Referencias

[1] Kernighan, B. W., Ritchie, D. M., El lenguaje de programación C, Prentice Hall, 1988.

[2] Harbison, S., Steele, G., C: A Reference Manual, 5th Edition, Prentice Hall, 2002.

[3] Tanenbaum, A., Structured Computer Organization, 6th Edition, Pearson, 2012.